

Special Online Edition for UVA Online Judge Users

Art of Programming Contest

C Programming | Data Structures | Algorithms

2nd Edition

AHMED SHAMSUL AREFIN

FOREWORD BY

MIGUEL A. REVILLA
UNIVERSITY OF VALLADOLID, SPAIN

REVIEWED BY

STEVEN HALIM
DR. M. LUTFAR RAHMAN

ACMSOLVER Training Series



An Essential Book for Programmers

ART OF PROGRAMMING CONTEST

C Programming Tutorials | Data Structures | Algorithms

Compiled by

Ahmed Shamsul Arefin

Graduate Student,
Institute of Information and Communication Technology
Bangladesh University of Engineering and Technology (BUET)
BSc. in Computer Science and Engineering, CUET

Reviewed By

Steven Halim

School of Computing, National University of Singapore
Singapore.

Dr. M. Lutfar Rahman

Professor, Department of Computer Science and Engineering
University of Dhaka.

Foreworded By

Professor Miguel A. Revilla

ACM-ICPC International Steering Committee Member and Problem Archivist
University of Valladolid,
Spain.

<http://acmicpc-live-archive.uva.es>
<http://online-judge.uva.es>



Gyankosh Prokashoni, Bangladesh

ISBN 984-32-3382-4

DEDICATED TO

Shahriar Manzoor

Judge ACM/ICPC World Finals 2003-2006

(Whose mails, posts and problems are invaluable to all programmers)

And

My loving parents and colleagues

ACKNOWLEDGEMENTS

I would like to thank following people for supporting me and helping me for the significant improvement of my humble works. Infact, this list is still incomplete.

Professor Miguel A. Revilla
Dr. M Kaykobad
Dr. M. Zafar Iqbal

Dr. M. Lutfar Rahman
Dr. Abu Taher
Howard Cheng
Steven Halim

Shahriar Manzoor
Carlos Marcelino Casas Cuadrado
Mahbub Murshed Suman
Salahuddin Mohammad Masum
Samiran Mahmud

M H Rasel

Sadiq M. Alam

Mehedi Bakht

Ahsan Raja Chowdhury
Mohammad Rubaiyat Ferdous Jewel
KM Hasan
Monirul Islam Sharif
Gahangir Hossain

S.M Saif Shams

Shah Md. Shamsul Alam

University of Valladolid, Spain.
North South University, Bangladesh
Shahjalal University of Science and Technology, Bangladesh
University of Dhaka, Bangladesh
Daffodil International University
University of Lethbridge, Canada
National University of Singapore, Singapore
South East University, Bangladesh
University of Valladolid, Spain
Arizona State University, USA
Daffodil International University
Dhaka University of Engineering and Technology
Chittagong University of Engineering and Technology
National University of Singapore, Singapore
Bangladesh University of Engineering and Technology
University of Dhaka
University of Toronto, Canada
North South University
Georgia Institute of Technology, USA
Chittagong University of Engineering and Technology
Shahjalal University of Science and Technology
Daffodil International University



Author's Biography: Ahmed Shamsul Arefin is completing his Masters from **Bangladesh University of Engineering & Technology (BUET)** and has completed BSc. in Computer Science and Engineering from **CUET**. In Computer Science and Engineering. He participated in the 2001 ACM Regional Contest in Dhaka, and his team was ranked 10th. He became contest organizer at Valladolid online judge by arranging "Rockford Programming Contest 2001" and local Contest at several universities. His Programming Contest Training Website "**ACMSolver.org**" has been linked with ACM UVA, USU and Polish Online Judge – Sphere.

His research interests are **Contests, Algorithms, Graph Theory** and **Web-based applications**. His Contact E-mail : asarefin@yahoo.com Web: <http://www.daffodilvarsity.edu.bd/acmsolver/asarefin/>

Preface to 2nd Edition

I am happy to be able to introduce the 2nd Edition of this book to the readers. The objective of this edition is not only to assist the contestants during the contest hours but also describing the core subjects of **Computer Science** such as **C Programming**, **Data Structures** and **Algorithms**. This edition is an improvement to the previous edition. Few more programming techniques like **STL (Standard Template Library)**, manipulating strings and handling mathematical functions are introduced here.

It is hoped that the new edition will be welcomed by all those for whom it is meant and this will become an **essential book for Computer Science students**.

Preface to 1st Edition

Why do programmers love Programming Contest? Because young computer programmers like to battle for fame, money, and they love algorithms. The first ACM-ICPC (International Collegiate Programming Contest) Asia Regional Contest Bangladesh was held at North South University in the year 1997. Except the year 2000, our country hosted this contest each year and our invaluable programmers have participated the world final every year from 1997.

Our performance in ACM/ICPC is boosting up day by day. The attention and time we are spending on solving moderate and difficult problems is noticeable. BUET, University of Dhaka, NSU and AIUB has produced many programmers who fought for World Finals. Institutions looking for boosting the performance of their teams in the programming contests may consider them as prospective coaches/trainers. Some universities have recently adopted another strategy. They are offering 1-credit courses for students interested in improving their problem-solving and programming skills.

I am very much grateful to our mentors, **Dr. M Kaykobad** who was honored with the “Best Coach” award in the World Finals in Honolulu. Under his dynamic presence our country teams became champion several times in the ACM/ICPC Asia Regional. **Dr. M. Zafar Iqbal**, Chief Judge of our ACM/ICPC Regional Contests. **Dr. Abul L Haque**, who first contacted **Dr. C.J. Hwang** (Asia Contests Director and Professor at Texas State University, San Marcos, USA) and wanted to have a n ACM/ICPC regional site at Dhaka back in 1997. Also a big thank should go to **Mr. Shahriar Manzoor**, our renown Problem Setter, Judging Director for ACM/ICPC Regional (Dhaka Site) and World Final Judge and Problem Setter. I would like to thank him personally because, he showed me the right way several times when I was setting problems for Valladolid Online Judge in “Rockford Programming Contest 2001” and while developing my Programming Contest Training Site “ACMSolver.org”.

Thanks to **Professor Miguel A. Revilla**, University of Valladolid, Spain for linking my ACMSolver (<http://www.acmsolver.org>) site with his world famous Valladolid Online Judge (<http://acm.uva.es/p>) and making me ACM Valladolid Online Judge Algorithmic Team Member for helping them to add some problems at live archive.

And also invaluable thanks to **Steven Halim**, a PhD Student of NUS, Singapore for the permission of using his website (<http://www.comp.nus.edu.sg/~stevenha/>) contents. A major part of this book is compiled from his renowned website. Of course, it is mentionable that his website is based upon USACO Training page located at (<http://ace.delos.com/>)

I am grateful to **Daffodil International University**, especially to honorable Vice-Chancellor **Professor Aminul Islam** and Dean, Faculty of Science and Informaion Technology **Dr. M. Lutfar Rahman** and all my colleagues at **Department of Computer Science and Engineering** here, for providing me the golden opportunity of doing something on ACM Programming Contest and other researches.

Furthermore, since this project is a collection of tutorials from several sources so all the authors of tutorials are acknowledged in the **Reference** section of this book. Tracking down the original authors of some of these tutorials is much difficult. I have tried to identify case by case and in each case asked permission. I apologize in advance if there are any oversights. If so, please let me know so that I can mention the name in future edition.

Finally I would like to add a line at the end of this preface, for last few years while making and maintaining my site on ACM Programming Contest, I have got few experiences. I felt that there should be some guideline for beginners to enter into the world of programming. So, I started collecting tutorials and compiling them to my site. Furthermore, this is another attempt to make Programming Contest in our country, as I have tried to put all my collections in a printed form. Your suggestions will be cordially accepted.

Best regards,

Ahmed Shamsul Arefin.



UNIVERSIDAD DE
VALLADOLID

Foreword Note

As the main responsible of the University of Valladolid Online Judge I has the feeling that this book is not only a recollection of tutorials as the author says in the preface, but also will be an essential part of the help sections of the UVa site, as it put together a lot of scattered information of the Online Judge, that may help to many programmers around the world, mainly to the newcomers, what is very important for us. The author proves a special interest in guiding the reader, and his tips must be considered almost as orders, as they are a result of a great experience as solver of problems as well as a problemsetter. Of course, the book is much more than an Online Judge user manual and contains very important information missing in our web, as the very interesting clasification of a lot of problems by categories, that analyze in detail and with examples. I think it is a book all our users should be allowed to access to, as is a perfect complement to our Online Judge.

Miguel A. Revilla

**ACM-ICPC International Steering Committee Member and Problem Archivist
University of Valladolid, Spain.**

<http://acmicpc-live-archive.uva.es>

<http://online-judge.uva.es>

University Of Dhaka



Review Note

A Computer programming contest is a pleasurable event for the budding programmers, but only a few books are available as a training manual for programming competitions.

This book is designed to serve as a textbook for an algorithm course focusing on programming as well as a programming course focusing on algorithms. The book is specially designed to train students to participate in competitions such as the ACM International Collegiate Programming Contest.

The book covers several important topics related to the development of programming skills such as, fundamental concepts of contest, game plan for a contest, essential data structures for contest, Input/output techniques, brute force method, mathematics, sorting, searching, greedy algorithms, dynamic programming, graphs, computational geometry, Valladolid Online Judge problem category, selected ACM programming problems, common codes/routines for programming, Standard Template Library (STL), PC² contest administration and team guide. The book also lists some important websites/books for ACM/ICPC Programmers.

I believe that the book will be of immense use for young programmers interested in taking part in programming competitions.

A handwritten signature in black ink, appearing to read 'M. Lutfar Rahman'.

Dr. M. Lutfar Rahman
Professor, Department of Computer Science and Engineering (CSE)
University of Dhaka.
Bangladesh.



Notes from Steven Halim

When I created my own website World of Seven few years back (<http://www.comp.nus.edu.sg/~stevenha>), my aim was to promote understanding of data structures and algorithms especially in the context of programming contest and to motivate more programmers to be more competitive by giving a lot of hints for many University of Valladolid (UVa) Online Judge problems. However, due to my busyness, I never managed to set aside a time to properly publicize the content of my website in a book format. Thus, I am glad that Ahmed compiled this book and he got my permission to do so. Hopefully, this book will be beneficial for the programmers in general, but especially to the Bangladeshi programmers where this book will be sold.

A handwritten signature in black ink, appearing to be 'SH', with a horizontal line extending to the right.

Steven Halim
National University of Singapore (NUS)
Singapore.

Contents

Chapter 1	Fundamental Concepts	14
Chapter 2	Game Plan For a Contest	19
Chapter 3	Programming In C: a Tutorial	27
Chapter 4	Essential Data Structures for Contest	72
Chapter 5	Input/Output Techniques	81
Chapter 6	Brute Force Method	85
Chapter 7	Mathematics	91
Chapter 8	Sorting	106
Chapter 9	Searching	113
Chapter 10	Greedy Algorithms	117
Chapter 11	Dynamic Programming	121
Chapter 12	Graphs	134
Chapter 13	Computational Geometry	172
Chapter 14	Valladolid OJ Problem Category	174
Appendix A	ACM Programming Problems	176
Appendix B	Common Codes/Routines For Programming	188
Appendix C	Standard Template Library (STL)	230
Appendix D	PC ² Contest Administration And Team Guide	235
Appendix E	Important Websites/Books for ACM Programmers	242



acm International Collegiate
Programming Contest

IBM | event
sponsor

What is the ACM Programming Contest?

The Association for Computing Machinery (ACM) sponsors a yearly programming contest, recently with the sponsorship of IBM. The contest is both well-known and highly regarded: last year 2400 teams competed from more than 100 nations competed at the regional levels. Sixty of these went on to the international finals. This contest is known as **ACM International Collegiate Programming Contest (ICPC)**.

The regional contest itself is typically held in November, with the finals in March. Teams of three students use C, C++, or Java to solve six to eight problems within five hours. One machine is provided to each team, leaving one or two team members free to work out an approach. Often, deciding which problems to attack first is the most important skill in the contest. The problems test the identification of underlying algorithms as much as programming savvy and speed.

CHAPTER 1 FUNDAMENTAL CONCEPTS

Programming Contest is a delightful playground for the exploration of intelligence of programmers. To start solving problems in contests, first of all, you have to fix your aim. Some contestants want to increase the number of problems solved by them and the other contestants want to solve less problems but with more efficiency. Choose any of the two categories and then start. A contestant without any aim can never prosper in 24 hours online judge contests. So, think about your aim.^[1]

If you are a beginner, first try to find the easier problems. Try to solve them within short time. At first, you may need more and more time to solve even simple problems. But do not be pessimistic. It is for your lack of practice. Try to solve easier problems as they increase your programming ability. Many beginners spend a lot of time for coding the program in a particular language but to be a great programmer you should not spend more times for coding, rather you should spend more time for debugging and thinking about the algorithm for the particular problem. A good programmer spends 10% time for coding and 45% time for thinking and the rest of the time for debugging. So to decrease the time for coding you should practice to solve easier problems first.

Do not try to use input file for input and even any output file for output when sending the program to online judges. All input and output parts should be done using standard input and outputs. If you are a C or C++ programmer try this, while coding and debugging for errors add the lines at the first line of the main procedure i.e.

```
#include <stdio.h>
main ()
{
    freopen("FILE_NAME_FOR_INPUT", "r", stdin);
    freopen("FILE_NAME_FOR OUTPUT", "w", stdout);
    Rest of the codes...
    return 0;}
```

But while sending to online judges remove the two lines with `freopen` to avoid restricted function errors. If you use the first `freopen` above, it will cause your program to take input from the file "FILE_NAME_FOR_INPUT". Write down the inputs in the file to avoid entering input several times for debugging. It saves a lot of time. But as the function opens input file which can be a cause of hacking the websites of online judges they don't allow using the function and if you use it they will give compilation error (Restricted Function). The second `freopen` is for generating the output of your program in a specified file named "FILE_NAME_FOR_OUTPUT" on the machine. It is very helpful when the output can't be justified just viewing the output window (Especially for String Manipulation Problem where even a single space character can be a cause of

Wrong answer). To learn about the function more check Microsoft Developer Network (MSDN Collection) and C programming helps.

Programming languages and dirty debugging

Most of the time a beginner faces this problem of deciding which programming language to be used to solve the problems. So, sometimes he uses such a programming language which he doesn't know deeply. That is why; he debugs for finding the faults for hour after hour and at last can understand that his problem is not in the algorithm, rather it is in the code written in that particular language. To avoid this, try to learn only one programming language very deeply and then to explore other flexible programming languages. The most commonly used languages are C, C++, PASCAL and JAVA. Java is the least used programming language among the other languages. Avoid dirty debugging.

Avoid Compilation Errors

The most common reply to the beginner from 24 hours online judge is COMPILATION ERROR (CE). The advices are,

- 1) When you use a function check the help and see whether it is available in Standard Form of the language. For example, do not use `strev` function of `string.h` header file of C and C++ as it is not ANSI C, C++ standard. You should make the function manually if you need it. Code manually or avoid those functions that are available in your particular compiler but not in Standard Form of the languages.
- 2) Don't use input and output file for your program. Take all the inputs for standard input and write all the outputs on standard output (normally on the console window). Check my previous topics.
- 3) Do not use `conio.h` header file in C or C++ as it is not available in Standard C and C++. Usually don't use any functions available in `<conio.h>` header file. It is the great cause of Compilation Error for the programmers that use Turbo C++ type compiler.
- 4) built-in functions and packages are not allowed for using in online judge.
- 5) Don't mail your program i.e. don't use yahoo, hotmail etc. for sending your program to judge as it is a complex method—write judge id, problems number etc. Rather use submit-system of the online judge for example, Submit page of Valladolid. Using the former will give you CE most of the time as they include there advertisements at the beginning and end of your program. So the judge can't recognize the extra characters concatenated in your sent code and gives you CE. About 90% CE we ever got is for this

reason. The mail system also breaks your programs into several lines causing Wrong Answer or Other Errors though your program was correct indeed.

There are many famous online judges that can judge your solution codes 24 hours. Some of them are:

- ▲ Valladolid OJ (<http://acm.uva.es/p>)
- ▲ Ural OJ (<http://acm.timus.ru>)
- ▲ Saratov OJ (<http://acm.sgu.ru>)
- ▲ ZJU OJ (<http://acm.zju.edu.cn>)
- ▲ Official ACM Live Archive (<http://cii-judge.baylor.edu/>)
- ▲ Peking University Online Judge (<http://acm.pku.edu.cn/JudgeOnline/>)
- ▲ Programming Challenges (<http://www.programming-challenges.com>)

Forget Efficiency and start solving easier problems

Sometimes, you may notice that many programmers solved many problems but they made very few submissions (they are geniuses!). At first, you may think that I should try to solve the problems as less try as possible. So, after solving a problem, you will not want to try it again with other algorithm (may be far far better than the previous algorithm you used to solve that problem) to update your rank in the rank lists. But my opinion is that if you think so you are in a wrong track. You should try other ways as in that and only that way you can know that which of the algorithms is better. Again in that way you will be able to know about various errors than can occur. If you don't submit, you can't know it. Perhaps a problem that you solved may be solved with less time in other way. So, my opinion is to try all the ways you know. In a word, if you are a beginner forget about efficiency.

Find the easier problems. Those problems are called ADHOC problems. You can find the list of those problems available in 24 OJ in S. Halim's, acmbeginner's, acmsolver's websites. Try to solve these problems and in that way you can increase your programming capability.

Learn algorithms

Most of the problems of Online Judges are dependent on various algorithms. An algorithm is a definite way to solve a particular problem. If you are now skilled in coding and solving easier problems, read the books of algorithms next. Of course, you should have a very good mathematical skill to understand various algorithms. Otherwise, there is no other way but just to skip the topics of the books. If you have skill in math, read the algorithms one by one, try to understand. After understanding the algorithms, try to write it in the programming language you have learnt (This is because, most of the

algorithms are described in Pseudocode). If you can write it without any errors, try to find the problems related to the algorithm, try to solve them. There are many famous books of algorithms. Try to make modified algorithm from the given algorithms in the book to solve the problems.

Use simple algorithms, that are guaranteed to solve the problem in question, even if they are not the optimum or the most elegant solution. Use them even if they are the most stupid solution, provided they work and they are not exponential. You are not competing for algorithm elegance or efficiency. You just need a correct algorithm, and you need it now. The simplest the algorithm, the more the chances are that you will code it correctly with your first shot at it.

This is the most important tip to follow in a programming contest. You don't have the time to design complex algorithms once you have an algorithm that will do your job. Judging on the size of your input you can implement the stupidest of algorithms and have a working solution in no time. Don't underestimate today's CPUs. A for loop of 10 million repetitions will execute in no time. And even if it takes 2 or 3 seconds you needn't bother. You just want a program that will finish in a couple of seconds. Usually the timeout for solutions is set to 30 seconds or more. Experience shows that if your algorithm takes more than 10 seconds to finish then it is probably exponential and you should do something better.

Obviously this tip should not be followed when writing critical code that needs to be as optimized as possible. However in my few years of experience we have only come to meet such critical code in device drivers. We are talking about routines that will execute thousands of times per second. In such a case every instruction counts. Otherwise it is not worth the while spending 5 hours for a 50% improvement on a routine that takes 10 milliseconds to complete and is called whenever the user presses a button. Nobody will ever notice the difference. Only you will know.

Simple Coding

1. Avoid the usage of the ++ or -- operators inside expressions or function calls. Always use them in a separate instruction. If you do this there is no chance that you introduce an error due to post-increment or pre-increment. Remember it makes no difference to the output code produced.
2. Avoid expressions of the form *p++.
3. Avoid pointer arithmetic. Instead of (p+5) use p[5].
4. Never code like :

```
return (x*y)+Func(t)/(1-s);
```

but like :

```
temp = func(t);  
RetVal = (x*y) + temp/(1-s);  
return RetVal;
```

This way you can check with your debugger what was the return value of Func(t) and what will be the return code of your function.

5. Avoid using the = operator. Instead of :

```
return (((x*8-111)%7)>5) ? y : 8-x;
```

Rather use :

```
Temp = ((x*8-111)%7);    if (5<Temp) return y; else return 8-x;
```

If you follow those rules then you eliminate all chances for trivial errors, and if you need to debug the code it will be much easier to do so.

- NAMING 1 : Don't use small and similar names for your variables. If you have three pointer variables don't name them p1, p2 and p3. Use descriptive names. Remember that your task is to write code that when you read it it says what it does. Using names like Index, RightMost, and Retries is much better than i, rm and rt. The time you waste by the extra typing is nothing compared to the gain of having a code that speaks for itself.
- NAMING 2 : Use hungarian naming, but to a certain extent. Even if you oppose it (which, whether you like it or not, is a sign of immaturity) it is of immense help.
- NAMING 3 : Don't use names like {i,j,k} for loop control variables. Use {I,K,M}. It is very easy to mistake a j for an i when you read code or "copy, paste & change" code, but there is no chance that you mistake I for K or M.

Last words

Practice makes a man perfect. So, try to solve more and more problems. A genius can't be built in a day. It is you who may be one of the first ten of the rank lists after someday. So, get a pc, install a programming language and start solving problem at once.^[1]

CHAPTER 2 GAME PLAN FOR A CONTEST

During a real time contest, teams consisting of three students and one computer are to solve as many of the given problems as possible within 5 hours. The team with the most problems solved wins, where "solved" means producing the right outputs for a set of (secret) test inputs. Though the individual skills of the team members are important, in order to be a top team it is necessary to make use of synergy within the team.^[2]

However, to make full use of a strategy, it is also important that your individual skills are as honed as possible. You do not have to be a genius as practicing can take you quite far. In our philosophy, there are three factors crucial for being a good programming team:

- ▲ Knowledge of standard algorithms and the ability to find an appropriate algorithm for every problem in the set;
- ▲ Ability to code an algorithm into a working program; and
- ▲ Having a strategy of cooperation with your teammates.

What is an Algorithm?

"A good algorithm is like a sharp knife - it does exactly what it is supposed to do with a minimum amount of applied effort. Using the wrong algorithm to solve a problem is trying to cut a steak with a screwdriver: you may eventually get a digestible result, but you will expend considerable more effort than necessary, and the result is unlikely to be aesthetically pleasing."

Algorithm is a step-by-step sequence of instructions for the computer to follow.

To be able to do something competitive in programming contests, you need to know a lot of well-known algorithms and ability to identify which algorithms is suitable for a particular problem (if the problem is straightforward), or which combinations or variants of algorithms (if the problem is a bit more complex).

A good and correct algorithm according to the judges in programming contests:

1. Must terminate
[otherwise: Time Limit/Memory Limit/Output Limit Exceeded will be given]
2. When it terminate, it must produce a correct output
[otherwise: the famous Wrong Answer reply will be given]
3. It should be as efficient as possible
[otherwise: Time Limit Exceeded will be given]

Ability to quickly identify problem types

In all programming contests, there are only three types of problems:

1. I haven't see this one before
2. I have seen this type before, but haven't or can't solve it
3. I have solve this type before

In programming contests, you will be dealing with a set of problems, not only one problem. The ability to quickly identify problems into the above mentioned contest-classifications (haven't see, have seen, have solved) will be one of key factor to do well in programming contests.

Mathematics	Prime Number
	Big Integer
	Permutation
	Number Theory
	Factorial
	Fibonacci
	Sequences
	Modulus
Dynmic Programming	Longest Common Subsequence
	Longest Increasing Subsequence
	Edit Distance
	0/1 Knapsack
	Coin Change
	Matrix Chain Multiplication
	Max Interval Sum
Graph	Traversal
	Flood Fill
	Floyed Warshal
	MST
	Max Bipertite Matching
	Network Flow
	Aritculation Point
Sorting	Bubble Sort
	Quick Sort
	Merge Sort (DAndC)
	Selection Sort
	Radix Sort
	Bucket Sort
Searching	Complete Search, Brute Force
	Binary Search (DAndC)
	BST
Simulation	Josephus
String Processing	String Matching
	Pattern Matching
Computational Geometry	Convex Hull
AdHoc	Trivial Problems

Sometimes, the algorithm may be 'nested' inside a loop of another algorithm. Such as binary search inside a DP algorithm, making the problem type identification not so trivial.

If you want to be able to compete well in programming contests, you must be able to know all that we listed above, with some precautions to ad-hoc problems.

'Ad Hoc' problems are those whose algorithms do not fall into standard categories with well-studied solutions. Each Ad Hoc problem is different... No specific or general techniques exist to solve them. This makes the problems the 'fun' ones (and sometimes frustrating), since each one presents a new challenge.

The solutions might require a novel data structure or an unusual set of loops or conditionals. Sometimes they require special combinations that are rare or at least rarely encountered. It usually requires careful problem description reading and usually yield to an attack that revolves around carefully sequencing the instructions given in the problem. Ad Hoc problems can still require reasonable optimizations and at least a degree of analysis that enables one to avoid loops nested five deep, for example.

Ability to analyze your algorithm

You have identified your problem. You think you know how to solve it. The question that you must ask now is simple: Given the maximum input bound (usually given in problem description), can my algorithm, with the complexity that I can compute, pass the time limit given in the programming contest.

Usually, there are more than one way to solve a problem. However, some of them may be incorrect and some of them is not fast enough. However, the rule of thumb is: **Brainstorm many possible algorithms - then pick the stupidest that works!**

Things to learn in algorithm

1. Proof of algorithm correctness (especially for Greedy algorithms)
2. Time/Space complexity analysis for non recursive algorithms.
3. For recursive algorithms, the knowledge of computing recurrence relations and analyze them: iterative method, substitution method, recursion tree method and finally, Master Theorem
4. Analysis of randomized algorithm which involves probabilistic knowledge, e.g: Random variable, Expectation, etc.
5. Amortized analysis.
6. Output-sensitive analysis, to analyze algorithm which depends on output size, example: $O(n \log k)$ LIS algorithm, which depends on k , which is output size not input size.

Table 1: Time comparison of different order of growth

We assume our computer can compute 1000 elements in 1 seconds (1000 ms)

Order of Growth	n	Time (ms)	Comment
$O(1)$	1000	1	Excellent, almost impossible for most cases
$O(\log n)$	1000	9.96	Very good, example: Binary Search
$O(n)$	1000	1000	Normal, Linear time algorithm
$O(n \log n)$	1000	9960	Average, this is usually found in sorting algorithm such as Quick sort
$O(n^2)$	1000	1000000	Slow
$O(n^3)$	1000	10^9	Slow, btw, All Pairs Shortest Paths algorithm: Floyd Warshall, is $O(N^3)$
$O(2^n)$	1000	2^{1000}	Poor, exponential growth... try to avoid this. Use Dynamic Programming if you can.
$O(n!)$	1000	uncountable	Typical NP-Complete problems.

Table 2: Limit of maximum input size under 60 seconds time limit (with assumptions)

Order of Growth	Time (ms)	Max Possible n	Comment
$O(1)$	60.000 ms	Virtually infinite	Best
$O(\log n)$	60.000 ms	$6^{18.000}$	A very very large number
$O(n)$	60.000 ms	60.000	Still a very big number
$O(n \log n)$	60.000 ms	~ 5.000	Moderate, average real life size
$O(n^2)$	60.000 ms	244	small
$O(n^3)$	60.000 ms	39	very small
$O(2^n)$	60.000 ms	16	avoid if you can
$O(n!)$	60.000 ms	8	extremely too small.

It may be useful to memorize the following ordering for quickly determine which algorithm perform better asymptotically: **constant** < **log n** < **n** < **n log n** < **n²** < **n³** < **2ⁿ** < **n!**

Some rules of thumb

1. Biggest built in data structure "long long" is $2^{63}-1$: $9 \cdot 10^{18}$ (up to 18 digits)
2. If you have k nested loops running about n iterations each, the program has $O(n^k)$ complexity
3. If your program is recursive with b recursive calls per level and has l levels, the program $O(b^l)$ complexity
4. Bear in mind that there are n! permutations and 2^n subsets or combinations of n elements when dealing with those kinds of algorithms

5. The best times for sorting n elements are $O(n \log n)$
6. DP algorithms which involves filling in a matrix usually in $O(n^3)$
7. In contest, most of the time $O(n \log n)$ algorithms will be sufficient.

The art of testing your code

You've done it. Identifying the problem, designing the best possible algorithm, you have calculate using time/space complexity, that it will be within time and memory limit given., and you have code it so well. But, is it 100% correct?

Depends on the programming contest's type, you may or may not get credit by solving the problem partially. In ACM ICPC, you will only get credit if your team's code solve all the test cases, that's it, you'll get either Accepted or Not Accepted (Wrong Answer, Time Limit Exceeded, etc). In IOI, there exist partial credit system, in which you will get score: number of correct/total number of test cases for each code that you submit.

In either case, you will need to be able to design a good, educated, tricky test cases. Sample input-output given in problem description is by default too trivial and therefore not a good way to measure your code's correctness for all input instances.

Rather than wasting your submission (and gaining time or points penalty) by getting wrong answer, you may want to design some tricky test cases first, test it in your own machine, and ensure your code is able to solve it correctly (otherwise, there is no point submitting your solution right?).

Some team coaches sometime ask their students to compete with each other by designing test cases. If student A's test cases can break other student's code, then A will get bonus point. You may want to try this in your school team training too.

Here is some guidelines in designing good test cases:

1. Must include sample input, the most trivial one, you even have the answer given.
2. Must include boundary cases, what is the maximum n, x, y , or other input variables, try varying their values to test for out of bound errors.
3. For multiple input test case, try using two identical test case consecutively. Both must output the same result. This is to check whether you forgot to initialize some variables, which will be easily identified if the first instance produce correct output but the second one doesn't.
4. Increase the size of input. Sometimes your program works for small input size, but behave wrongly when input size increases.
5. Tricky test cases, analyze the problem description and identify parts that are tricky, test them to your code.
6. Don't assume input will always nicely formatted if the problem description didn't say

so. Try inserting white spaces (space, tabs) in your input, check whether your code is able to read in the values correctly

7. Finally, do random test cases, try random input and check your code's correctness.

Producing Winning Solution

A good way to get a competitive edge is to write down a game plan for what you're going to do in a contest round. This will help you script out your actions, in terms of what to do both when things go right and when things go wrong. This way you can spend your thinking time in the round figuring out programming problems and not trying to figure out what the heck you should do next... it's sort of like Pre-Computing your reactions to most situations.

Read through all the problems first, don't directly attempt one problem since you may missed easier problem.

1. Order the problems: shortest job first, in terms of your effort (shortest to longest: done it before, easy, unfamiliar, hard).
2. Sketch the algorithms, complexity, the numbers, data structures, tricky details.
3. Brainstorm other possible algorithms (if any) - then pick the stupidest that works!
4. Do the Math! (space & time complexity & plug-in actual expected & worst case numbers).
5. Code it of course, as fast as possible, and it must be correct.
6. Try to break the algorithm - use special (degenerate?) test cases.

Coding a problem

1. Only coding after you finalize your algorithm.
2. Create test data for tricky cases.
3. Code the input routine and test it (write extra output routines to show data).
4. Code the output routine and test it.
5. Write data structures needed.
6. Stepwise refinement: write comments outlining the program logic.
7. Fill in code and debug one section at a time.
8. Get it working & verify correctness (use trivial test cases).
9. Try to break the code - use special cases for code correctness.

Time management strategy and "damage control" scenarios

Have a plan for what to do when various (foreseeable!) things go wrong; imagine problems you might have and figure out how you want to react. The central question is:

"When do you spend more time debugging a program, and when do you cut your losses and move on?". Consider these issues:

1. How long have you spent debugging it already?
2. What type of bug do you seem to have?
3. Is your algorithm wrong?
4. Do your data structures need to be changed?
5. Do you have any clue about what's going wrong?
6. A short amount (20 mins) of debugging is better than switching to anything else; but you might be able to solve another from scratch in 45 mins.
7. When do you go back to a problem you've abandoned previously?
8. When do you spend more time optimizing a program, and when do you switch?
9. Consider from here out - forget prior effort, focus on the future: how can you get the most points in the next hour with what you have?

Tips & tricks for contests

1. Brute force when you can, Brute force algorithm tends to be the easiest to implement.
2. KISS: Simple is smart! (Keep It Simple, Stupid !!! / Keep It Short & Simple).
3. Hint: focus on limits (specified in problem statement).
4. Waste memory when it makes your life easier (trade memory space for speed).
5. Don't delete your extra debugging output, comment it out.
6. Optimize progressively, and only as much as needed.
7. Keep all working versions!
8. Code to debug:
 - a. white space is good,
 - b. use meaningful variable names,
 - c. don't reuse variables, (we are not doing software engineering here)
 - d. stepwise refinement,
 - e. Comment before code.
9. Avoid pointers if you can.
10. Avoid dynamic memory like the plague: statically allocate everything. (yeah yeah)
11. Try not to use floating point; if you have to, put tolerances in everywhere (never test equality)
12. Comments on comments:
 - a. Not long prose, just brief notes.
 - b. Explain high-level functionality: `++i; /* increase the value of i by */` is worse than useless.
 - c. Explain code trickery.
 - d. Delimit & document functional sections.

Keep a log of your performance in each contest: successes, mistakes, and what you could have done better; use this to rewrite and improve your game plan!

The Judges Are Evil and Out to Get You

Judges don't want to put easy problems on the contest, because they have thought up too many difficult problems. So what we do is hide the easy problems, hoping that you will be tricked into working on the harder ones. If we want you to add two non-negative numbers together, there will be pages of text on the addition of '0' to the number system and 3D-pictures to explain the process of addition as it was imagined on some island that nobody has ever heard of.

Once we've scared you away from the easy problems, we make the hard ones look easy. 'Given two polygons, find the area of their intersection.' Easy, right?

It isn't always obvious that a problem is easy, so teams ignore the problems or start on overly complex approaches to them. Remember, there are dozens of other teams working on the same problems, and they will help you find the easy problems. **If everyone is solving problem G, maybe you should take another look at it.**^[6]

CHAPTER 3 PROGRAMMING IN C: A TUTORIAL^[11]

C was created by **Dennis Ritchie** at the **Bell Telephone Laboratories** in 1972. Because C is such a powerful and flexible language, its use quickly spread beyond Bell Labs. Programmers everywhere began using it to write all sorts of programs. Soon, however, different organizations began utilizing their own versions of C, and subtle differences between implementations started to cause programmers headaches. In response to this problem, the **American National Standards Institute (ANSI)** formed a committee in 1983 to establish a standard definition of C, which became known as ANSI Standard C. With few exceptions, every modern C compiler has the ability to adhere this standard [11].

A Simple C Program

A C program consists of one or more functions, which are similar to the functions and subroutines of a Fortran program or the procedures of PL/I, and perhaps some external data definitions. `main` is such a function, and in fact all C programs must have a `main`. Execution of the program begins at the first statement of `main`. `main` will usually invoke other functions to perform its job, some coming from the same program, and others from libraries.

```
main( ) {  
    printf("hello, world");  
}
```

`printf` is a library function which will format and print output on the terminal (unless some other destination is specified). In this case it prints

```
hello, world
```

A Working C Program; Variables; Types and Type Declarations

Here's a bigger program that adds three integers and prints their sum.

```
main( ) {  
    int a, b, c, sum;  
    a = 1;  b = 2;  c = 3;  
    sum = a + b + c;  
    printf("sum is %d", sum);  
}
```

Arithmetic and the assignment statements are much the same as in Fortran (except for the semicolons) or PL/I. The format of C programs is quite free. We can put several

statements on a line if we want, or we can split a statement among several lines if it seems desirable. The split may be between any of the operators or variables, but *not* in the middle of a name or operator. As a matter of style, spaces, tabs, and newlines should be used freely to enhance readability.

C has fundamental types of variables:

Variable Type	Keyword	Bytes Required	Range
Character	char	1	-128 to 127
Integer	int	2	-32768 to 32767
Short integer	short	2	-32768 to 32767
Long integer	long	4	-2,147,483,648 to 2,147,438,647
Unsigned character	unsigned char	1	0 to 255
Unsigned integer	unsigned int	2	0 to 65535
Unsigned short integer	unsigned short	2	0 to 65535
Unsigned long integer	unsigned long	4	0 to 4,294,967,295
Single-precision	float	4	1.2E-38 to
floating-point			3.4E38
Double-precision	double	8	2.2E-308 to
floating-point			1.8E308 ²

There are also arrays and structures of these basic types, pointers to them and functions that return them, all of which we will meet shortly.

All variables in a C program must be declared, although this can sometimes be done implicitly by context. Declarations must precede executable statements. The declaration

```
int a, b, c, sum;
```

Variable names have one to eight characters, chosen from A-Z, a-z, 0-9, and `_`, and start with a non-digit.

Constants

We have already seen decimal integer constants in the previous example-- 1, 2, and 3. Since C is often used for system programming and bit-manipulation, octal numbers are an important part of the language. In C, any number that begins with 0 (zero!) is an octal integer (and hence can't have any 8's or 9's in it). Thus `0777` is an octal constant, with decimal value 511.

A ``character" is one byte (an inherently machine-dependent concept). Most often this is expressed as a character constant, which is one character enclosed in single quotes. However, it may be any quantity that fits in a byte, as in flags below:

```
char quest, newline, flags;
quest = '?';
newline = '\n';
flags = 077;
```

The sequence ``\n' is C notation for ``newline character", which, when printed, skips the terminal to the beginning of the next line. Notice that ``\n' represents only a single character. There are several other ``escapes" like ``\n' for representing hard-to-get or invisible characters, such as ``\t' for tab, ``\b' for backspace, ``\0' for end of file, and ``\\' for the backslash itself.

Simple I/O – `getchar()`, `putchar()`, `printf ()`

getchar and **putchar** are the basic I/O library functions in C. `getchar` fetches one character from the standard input (usually the terminal) each time it is called, and returns that character as the value of the function. When it reaches the end of whatever file it is reading, thereafter it returns the character represented by ``\0' (ascii NUL, which has value zero). We will see how to use this very shortly.

```
main( ) {
    char c;
    c = getchar( );
    putchar(c);
}
```

`putchar` puts one character out on the standard output (usually the terminal) each time it is called. So the program above reads one character and writes it back out. By itself, this isn't very interesting, but observe that if we put a loop around this, and add a test for end of file, we have a complete program for copying one file to another.

`printf` is a more complicated function for producing formatted output.

```
printf ("hello, world\n");
```

is the simplest use. The string ``hello, world\n" is printed out.

More complicated, if `sum` is 6,

```
printf ("sum is %d\n", sum);
```

prints

```
sum is 6
```

Within the first argument of `printf`, the characters ```%d"` signify that the next argument in the argument list is to be printed as a base 10 number.

Other useful formatting commands are ```%c"` to print out a single character, ```%s"` to print out an entire string, and ```%o"` to print a number as octal instead of decimal (no leading zero). For example,

```
n = 511;
printf ("What is the value of %d in octal?", n);
printf ("%s! %d decimal is %o octal\n", "Right", n, n);
```

prints

```
What is the value of 511 in octal? Right! 511 decimal
is 777 octal
```

If – relational operators and compound statements

The basic conditional-testing statement in C is the `if` statement:

```
c = getchar( );
if( c == '?' )
    printf("why did you type a question mark?\n");
```

The simplest form of **`if`** is

```
if (expression) statement
```

The condition to be tested is any expression enclosed in parentheses. It is followed by a statement. The expression is evaluated, and if its value is non-zero, the statement is executed. There's an optional **`else`** clause, to be described soon.

The character sequence `'=='` is one of the relational operators in C; here is the complete set:

```
==      equal to (.EQ. to Fortraners)
!=      not equal to
>       greater than
```

```
<      less than
>=     greater than or equal to
<=     less than or equal to
```

The value of "`expression relation expression`" is 1 if the relation is true, and 0 if false. Don't forget that the equality test is `==`; a single `=` causes an assignment, not a test, and invariably leads to disaster.

Tests can be combined with the operators `&&` (AND), `||` (OR), and `!` (NOT). For example, we can test whether a character is blank or tab or newline with

```
if( c==' ' || c=='\t' || c=='\n' ) ...
```

C guarantees that `&&` and `||` are evaluated left to right -- we shall soon see cases where this matters.

As a simple example, suppose we want to ensure that `a` is bigger than `b`, as part of a sort routine. The interchange of `a` and `b` takes three statements in C, grouped together by `{}`:

```
if (a < b) {
    t = a;
    a = b;
    b = t;
}
```

As a general rule in C, anywhere you can use a simple statement, you can use any compound statement, which is just a number of simple or compound ones enclosed in `{}`. There is no semicolon after the `}` of a compound statement, but there *is* a semicolon after the last non-compound statement inside the `{}`.

While Statement; Assignment within an Expression; Null Statement

The basic looping mechanism in C is the while statement. Here's a program that copies its input to its output a character at a time. Remember that `'\0'` marks the end of file.

```
main( ) {
    char c;
    while( (c=getchar( )) != '\0' )
        putchar(c);
}
```

The while statement is a loop, whose general form is

```
while (expression) statement
```

Its meaning is

- (a) evaluate the expression
- (b) if its value is true (i.e., not zero) do the statement, and go back to (a)

Because the expression is tested before the statement is executed, the statement part can be executed zero times, which is often desirable. As in the if statement, the expression and the statement can both be arbitrarily complicated, although we haven't seen that yet. Our example gets the character, assigns it to `c`, and then tests if it's a `'\0'`. If it is not a `'\0'`, the statement part of the while is executed, printing the character. The while then repeats. When the input character is finally a `'\0'`, the while terminates, and so does main.

Notice that we used an assignment statement

```
c = getchar( )
```

within an expression. This is a handy notational shortcut which often produces clearer code. (In fact it is often the only way to write the code cleanly. As an exercise, rewrite the file-copy without using an assignment inside an expression.) It works because an assignment statement has a value, just as any other expression does. Its value is the value of the right hand side. This also implies that we can use multiple assignments like

```
x = y = z = 0;
```

Evaluation goes from right to left.

By the way, the extra parentheses in the assignment statement within the conditional were really necessary: if we had said

```
c = getchar( ) != '\0'
```

`c` would be set to 0 or 1 depending on whether the character fetched was an end of file or not. This is because in the absence of parentheses the assignment operator `'='` is evaluated after the relational operator `'!='`. When in doubt, or even if not, parenthesize.

```
main( ) {  
    while( putchar(getchar( )) != '\0' ) ;  
}
```

What statement is being repeated? None, or technically, the null statement, because all the work is really done within the test part of the while. This version is slightly different from the previous one, because the final `\0` is copied to the output before we decide to stop.

Arithmetic

The arithmetic operators are the usual `+`, `-`, `*`, and `/` (truncating integer division if the operands are both int), and the remainder or mod operator `%`:

```
x = a % b;
```

sets `x` to the remainder after `a` is divided by `b` (i.e., `a mod b`). The results are machine dependent unless `a` and `b` are both positive.

In arithmetic, char variables can usually be treated like int variables. Arithmetic on characters is quite legal, and often makes sense:

```
c = c + 'A' - 'a';
```

converts a single lower case ascii character stored in `c` to upper case, making use of the fact that corresponding ascii letters are a fixed distance apart. The rule governing this arithmetic is that all chars are converted to int before the arithmetic is done. Beware that conversion may involve sign-extension if the leftmost bit of a character is 1, the resulting integer might be negative. (This doesn't happen with genuine characters on any current machine.)

So to convert a file into lower case:

```
main( ) {
    char c;
    while( (c=getchar( )) != '\0' )
        if( 'A' <= c && c <= 'Z' )
            putchar(c + 'a' - 'A');
        else
            putchar(c); }
```

Else Clause; Conditional Expressions

We just used an `else` after an `if`. The most general form of `if` is

```
if (expression) statement1 else statement2
```

the `else` part is optional, but often useful. The canonical example sets `x` to the minimum of `a` and `b`:

```
if (a < b)
    x = a;
else
    x = b;
```

C provides an alternate form of conditional which is often more concise. It is called the "conditional expression" because it is a conditional which actually has a value and can be used anywhere an expression can. The value of

```
a < b ? a : b;
```

is `a` if `a` is less than `b`; it is `b` otherwise. In general, the form

```
expr1 ? expr2 : expr3
```

To set `x` to the minimum of `a` and `b`, then:

```
x = (a < b ? a : b);
```

The parentheses aren't necessary because `'?:'` is evaluated before `'='`, but safety first.

Going a step further, we could write the loop in the lower-case program as

```
while( (c=getchar( )) != '\0' )
    putchar( ('A'<=c && c<='Z') ? c-'A'+'a' : c );
```

If's and else's can be used to construct logic that branches one of several ways and then rejoins, a common programming structure, in this way:

```
if(...)
    {...}
else if(...)
    {...}
else if(...)
    {...}
else
    {...}
```

The conditions are tested in order, and exactly one block is executed; either the first one whose if is satisfied, or the one for the last `else`. When this block is finished, the next statement executed is the one after the last `else`. If no action is to be taken for the "default" case, omit the last `else`.

For example, to count letters, digits and others in a file, we could write

```
main( ) {
    int let, dig, other, c;
    let = dig = other = 0;
    while( (c=getchar( )) != '\0' )
        if( ('A'<=c && c<='Z') || ('a'<=c && c<='z') )
            ++let;
        else if( '0'<=c && c<='9' ) ++dig;
        else ++other;
    printf("%d letters, %d digits, %d others\n", let, dig, other);
}
```

This code letters, digits and others in a file.

Increment and Decrement Operators

In addition to the usual '-', C also has two other interesting unary operators, '++' (increment) and '--' (decrement). Suppose we want to count the lines in a file.

```
main( ) {
    int c,n;
    n = 0;
    while( (c=getchar( )) != '\0' )
        if( c == '\n' )
            ++n;
    printf("%d lines\n", n);
}
```

++n is equivalent to n=n+1 but clearer, particularly when n is a complicated expression. '++' and '--' can be applied only to int's and char's (and pointers which we haven't got to yet).

The unusual feature of '++' and '--' is that they can be used either before or after a variable. The value of ++k is the value of k *after* it has been incremented. The value of k++ is k *before* it is incremented. Suppose k is 5. Then

```
x = ++k;
```

increments k to 6 and then sets x to the resulting value, i.e., to 6. But

```
x = k++;
```

first sets *x* to 5, and *then* increments *k* to 6. The incrementing effect of *++k* and *k++* is the same, but their values are respectively 5 and 6. We shall soon see examples where both of these uses are important.

Arrays

In C, as in Fortran or PL/I, it is possible to make arrays whose elements are basic types. Thus we can make an array of 10 integers with the declaration

```
int x[10];
```

The square brackets mean subscripting; parentheses are used only for function references. Array indexes begin at zero, so the elements of *x* are

```
x[0], x[1], x[2], ..., x[9]
```

If an array has *n* elements, the largest subscript is *n*-1.

Multiple-dimension arrays are provided, though not much used above two dimensions. The declaration and use look like

```
int name[10] [20];  
n = name[i+j] [1] + name[k] [2];
```

Subscripts can be arbitrary integer expressions. Multi-dimension arrays are stored by row (opposite to Fortran), so the rightmost subscript varies fastest; *name* has 10 rows and 20 columns.

Here is a program which reads a line, stores it in a buffer, and prints its length (excluding the newline at the end).

```
main( ) {  
    int n, c;  
    char line[100];  
    n = 0;  
    while( (c=getchar( )) != '\n' ) {  
        if( n < 100 )  
            line[n] = c;  
        n++;  
    }  
    printf("length = %d\n", n);  
}
```

As a more complicated problem, suppose we want to print the count for each line in the input, still storing the first 100 characters of each line. Try it as an exercise before looking at the solution:

```
main( ) {
    int n, c; char line[100];
    n = 0;
    while( (c=getchar( )) != '\0' )
        if( c == '\n' ) {
            printf("%d", n);
            n = 0;
        }
        else {
            if( n < 100 ) line[n] = c;
            n++;
        }
}
```

Above code stores first 100 characters of each line!

Character Arrays; Strings

Text is usually kept as an array of characters, as we did with `line[ ]` in the example above. By convention in C, the last character in a character array should be a `'\0'` because most programs that manipulate character arrays expect it. For example, `printf` uses the `'\0'` to detect the end of a character array when printing it out with a `'%s'`.

We can copy a character array `s` into another `t` like this:

```
i = 0;
while( (t[i]=s[i]) != '\0' )
    i++;
```

Most of the time we have to put in our own `'\0'` at the end of a string; if we want to print the line with `printf`, it's necessary. This code prints the character count before the line:

```
main( ) {
    int n;
    char line[100];
    n = 0;
    while( (line[n++]=getchar( )) != '\n' );
    line[n] = '\0';
    printf("%d:\t%s", n, line);
}
```

Here we increment *n* in the subscript itself, but only after the previous value has been used. The character is read, placed in *line[n]*, and only then *n* is incremented.

There is one place and one place only where C puts in the `'\0'` at the end of a character array for you, and that is in the construction

```
"stuff between double quotes"
```

The compiler puts a `'\0'` at the end automatically. Text enclosed in double quotes is called a *string*; its properties are precisely those of an (initialized) array of characters.

for Statement

The for statement is a somewhat generalized while that lets us put the initialization and increment parts of a loop into a single statement along with the test. The general form of the for is

```
for( initialization; expression; increment )  
    statement
```

The meaning is exactly

```
initialization;  
while( expression ) {  
    statement  
    increment;  
}
```

This slightly more ornate example adds up the elements of an array:

```
sum = 0;  
for( i=0; i<n; i++)sum = sum + array[i];
```

In the for statement, the initialization can be left out if you want, but the semicolon has to be there. The increment is also optional. It is *not* followed by a semicolon. The second clause, the test, works the same way as in the `while`: if the expression is true (not zero) do another loop, otherwise get on with the next statement. As with the `while`, the for loop may be done zero times. If the expression is left out, it is taken to be always true, so

```
for( ; ; ) ...  
while( 1 ) ...
```

are both infinite loops.

You might ask why we use a `for` since it's so much like a `while`. (You might also ask why we use a `while` because...) The `for` is usually preferable because it keeps the code where it's used and sometimes eliminates the need for compound statements, as in this code that zeros a two-dimensional array:

```
for( i=0; i<n; i++ )
    for( j=0; j<m; j++ )
        array[i][j] = 0;
```

Functions; Comments

Suppose we want, as part of a larger program, to count the occurrences of the ascii characters in some input text. Let us also map illegal characters (those with value >127 or <0) into one pile. Since this is presumably an isolated part of the program, good practice dictates making it a separate function. Here is one way:

```
main( ) {
    int hist[129];      /* 128 legal chars + 1 illegal group */
    ...
    count(hist, 128);   /* count the letters into hist */
    printf( ... );      /* comments look like this; use them */
    /* anywhere blanks, tabs or newlines could appear */
}

count(buf, size)
    int size, buf[ ]; {
    int i, c;
    for( i=0; i<=size; i++ )
        buf[i] = 0;      /* set buf to zero */
    while( (c=getchar( )) != '\0' ) { /* read til eof */
        if( c > size || c < 0 )
            c = size;     /* fix illegal input */
        buf[c]++;
    }
    return;
}
```

We have already seen many examples of calling a function, so let us concentrate on how to define one. Since `count` has two arguments, we need to declare them, as shown, giving their types, and in the case of `buf`, the fact that it is an array. The declarations of arguments go between the argument list and the opening `{`. There is no need to specify the size of the array `buf`, for it is defined outside of `count`.

The return statement simply says to go back to the calling routine. In fact, we could have omitted it, since a return is implied at the end of a function.

What if we wanted count to return a value, say the number of characters read? The return statement allows for this too:

```
int i, c, nchar;
nchar = 0;
...
while( (c=getchar( )) != '\0' ) {
    if( c > size || c < 0 )
        c = size;
    buf[c]++;
    nchar++;
}
return(nchar);
```

Any expression can appear within the parentheses. Here is a function to compute the minimum of two integers:

```
min(a, b)
int a, b; {
    return( a < b ? a : b );
}
```

To copy a character array, we could write the function

```
strcpy(s1, s2)          /* copies s1 to s2 */
char s1[ ], s2[ ]; {
    int i;
    for( i = 0; (s2[i] = s1[i]) != '\0'; i++ ); }
```

As is often the case, all the work is done by the assignment statement embedded in the test part of the for. Again, the declarations of the arguments s1 and s2 omit the sizes, because they don't matter to strcpy. (In the section on pointers, we will see a more efficient way to do a string copy.)

There is a subtlety in function usage which can trap the unsuspecting Fortran programmer. Simple variables (not arrays) are passed in C by "call by value", which means that the called function is given a copy of its arguments, and doesn't know their addresses. This makes it impossible to change the value of one of the actual input arguments.

There are two ways out of this dilemma. One is to make special arrangements to pass to the function the address of a variable instead of its value. The other is to make the variable

a global or external variable, which is known to each function by its name. We will discuss both possibilities in the next few sections.

Local and External Variables

If we say

```
f( ) {  
    int x;  
    ...  
}  
g( ) {  
    int x;  
    ...  
}
```

each *x* is *local* to its own routine -- the *x* in *f* is unrelated to the *x* in *g*. (Local variables are also called "automatic".) Furthermore each local variable in a routine appears only when the function is called, and *disappears* when the function is exited. Local variables have no memory from one call to the next and must be explicitly initialized upon each entry. (There is a static storage class for making local variables with memory; we won't discuss it.)

As opposed to local variables, external variables are defined external to all functions, and are (potentially) available to all functions. External storage always remains in existence. To make variables external we have to define them external to all functions, and, wherever we want to use them, make a declaration.

```
main( ) {  
    extern int nchar, hist[ ];  
    ...  
    count( );  
    ...  
}  
count( ) {  
    extern int nchar, hist[ ];  
    int i, c;  
    ...  
}  
  
int    hist[129];    /* space for histogram */  
int    nchar;        /* character count */
```

Roughly speaking, any function that wishes to access an external variable must contain an *extern* declaration for it. The declaration is the same as others, except for the added keyword *extern*. Furthermore, there must somewhere be a definition of the external variables external to all functions.

External variables can be initialized; they are set to zero if not explicitly initialized. In its simplest form, initialization is done by putting the value (which must be a constant) after the definition:

Pointers

A pointer in C is the address of something. It is a rare case indeed when we care what the specific address itself is, but pointers are a quite common way to get at the contents of something. The unary operator `&` is used to produce the address of an object, if it has one. Thus

```
int a, b;  
b = &a;
```

puts the address of `a` into `b`. We can't do much with it except print it or pass it to some other routine, because we haven't given `b` the right kind of declaration. But if we declare that `b` is indeed a pointer to an integer, we're in good shape:

```
int a, *b, c;  
b = &a;  
c = *b;
```

`b` contains the address of `a` and `*c = *b` means to use the value in `b` as an address, i.e., as a pointer. The effect is that we get back the contents of `a`, albeit rather indirectly. (It's always the case that `*&x` is the same as `x` if `x` has an address.)

The most frequent use of pointers in C is for walking efficiently along arrays. In fact, in the implementation of an array, the array name represents the address of the zeroth element of the array, so you can't use it on the left side of an expression. (You can't change the address of something by assigning to it.) If we say

```
char *y; char x[100];
```

`y` is of type pointer to character (although it doesn't yet point anywhere). We can make `y` point to an element of `x` by either of

```
y = &x[0];  
y = x;
```

Since `x` is the address of `x[0]` this is legal and consistent.

Now `*y` gives `x[0]`. More importantly,

```
*(y+1) gives x[1]  
*(y+i) gives x[i]
```


and the sequence

```
y = &x[0];  
y++;
```

leaves `y` pointing at `x[1]`.

Let's use pointers in a function `length` that computes how long a character array is. Remember that by convention all character arrays are terminated with a `'\0'`. (And if they aren't, this program will blow up inevitably.) The old way:

```
length(s)  
    char s[ ]; {  
        int n;  
        for( n=0; s[n] != '\0'; )  
            n++;  
        return(n);  
    }
```

Rewriting with pointers gives

```
length(s)  
    char *s; {  
        int n;  
        for( n=0; *s != '\0'; s++ )  
            n++;  
        return(n);    }
```

You can now see why we have to say what kind of thing `s` points to -- if we're to increment it with `s++` we have to increment it by the right amount.

The pointer version is more efficient (this is almost always true) but even more compact is

```
for( n=0; *s++ != '\0'; n++ );
```

The `'*s'` returns a character; the `'++'` increments the pointer so we'll get the next character next time around. As you can see, as we make things more efficient, we also make them less clear. But `'*s++'` is an idiom so common that you have to know it.

Going a step further, here's our function `strcpy` that copies a character array `s` to another `t`.

```
strcpy(s,t)  
    char *s, *t; {  
        while(*t++ = *s++);  
    }
```

We have omitted the test against `'\0'`, because `'\0'` is identically zero; you will often see the code this way.

For arguments to a function, and there only, the declarations

```
char s[ ];  
char *s;
```

are equivalent -- a pointer to a type, or an array of unspecified size of that type, are the same thing.

Function Arguments

Look back at the [function `strcpy` in the previous section](#). We passed it two string names as arguments, then proceeded to clobber both of them by incrementation. So how come we don't lose the original strings in the function that called `strcpy`?

As we said before, C is a "call by value" language: when you make a function call like `f(x)`, the *value* of `x` is passed, not its address. So there's no way to *alter* `x` from inside `f`. If `x` is an array (`char x[10]`) this isn't a problem, because `x` *is* an address anyway, and you're not trying to change it, just what it addresses. This is why `strcpy` works as it does. And it's convenient not to have to worry about making temporary copies of the input arguments.

But what if `x` is a scalar and you do want to change it? In that case, you have to pass the *address* of `x` to `f`, and then use it as a pointer. Thus for example, to interchange two integers, we must write

```
flip(x, y)  
{  
    int *x, *y; {  
        int temp;  
        temp = *x;  
        *x = *y;  
        *y = temp;  
    }  
}
```

and to call `flip`, we have to pass the addresses of the variables:

```
flip (&a, &b);
```

Which interchange two integers.

The Switch Statement ; Break ; Continue

The switch statement can be used to replace the multi-way test we used in the last example. When the tests are like this:

```
if( c == 'a' ) ...
else if( c == 'b' ) ...
else if( c == 'c' ) ...
else ...
```

testing a value against a series of constants, the switch statement is often clearer and usually gives better code. Use it like this:

```
switch( c ) {

case 'a':
    aflag++;
    break;
case 'b':
    bflag++;
    break;
case 'c':
    cflag++;
    break;
default:
    printf("%c?\n", c);
    break;
}
```

The case statements label the various actions we want; `default` gets done if none of the other cases are satisfied. (A `default` is optional; if it isn't there, and none of the cases match, you just fall out the bottom.)

The `break` statement in this example is new. It is there because the cases are just labels, and after you do one of them, you fall through to the next unless you take some explicit action to escape. This is a mixed blessing. On the positive side, you can have multiple cases on a single statement; we might want to allow both upper and lower

But what if we just want to get out after doing case 'a' ? We could get out of a `case` of the `switch` with a `goto`, but this is really ugly. The `break` statement lets us exit without either `goto` or `label`.

The `break` statement also works in `for` and `while` statements; it causes an immediate exit from the loop.

The `continue` statement works *only* inside `for`'s and `while`'s; it causes the next iteration of the loop to be started. This means it goes to the increment part of the `for` and the test part of the `while`.

Structures

The main use of structures is to lump together collections of disparate variable types, so they can conveniently be treated as a unit. For example, if we were writing a compiler or assembler, we might need for each identifier information like its name (a character array), its source line number (an integer), some type information (a character, perhaps), and probably a usage count (another integer).

```
char    id[10];
int     line;
char    type;
int     usage;
```

We can make a structure out of this quite easily. We first tell C what the structure will look like, that is, what kinds of things it contains; after that we can actually reserve storage for it, either in the same statement or separately. The simplest thing is to define it and allocate storage all at once:

```
struct {
    char    id[10];
    int     line;
    char    type;
    int     usage;      } sym;
```

This defines `sym` to be a structure with the specified shape; `id`, `line`, `type` and `usage` are members of the structure. The way we refer to any particular member of the structure is

```
structure-name . member
```

as in

```
sym.type = 077;
if( sym.usage == 0 ) ...
while( sym.id[j++] ) ...
    etc.
```

Although the names of structure members never stand alone, they still have to be unique; there can't be another `id` or `usage` in some other structure.

So far we haven't gained much. The advantages of structures start to come when we have arrays of structures, or when we want to pass complicated data layouts between functions. Suppose we wanted to make a symbol table for up to 100 identifiers. We could extend our definitions like

```
char    id[100][10];
int     line[100];
char    type[100];
int     usage[100];
```

but a structure lets us rearrange this spread-out information so all the data about a single identifier is collected into one lump:

```
struct {
    char    id[10];
    int     line;
    char    type;
    int     usage;
} sym[100];
```

This makes `sym` an array of structures; each array element has the specified shape. Now we can refer to members as

```
sym[i].usage++; /* increment usage of i-th identifier */
for( j=0; sym[i].id[j++] != '\0'; ) ...
etc.
```

Thus to print a list of all identifiers that haven't been used, together with their line number,

```
for( i=0; i<nsym; i++ )
    if( sym[i].usage == 0 )
        printf("%d\t%s\n", sym[i].line, sym[i].id);
```

Suppose we now want to write a function `lookup(name)` which will tell us if `name` already exists in `sym`, by giving its index, or that it doesn't, by returning a -1. We can't pass a structure to a function directly; we have to either define it externally, or pass a pointer to it. Let's try the first way first.

```
int     nsym    0;      /* current length of symbol table */

struct {
    char    id[10];
    int     line;
    char    type;
    int     usage;
} sym[100];             /* symbol table */
```

```

main( ) {
    ...
    if( (index = lookup(newname)) >= 0 )
        sym[index].usage++;          /* already there ... */
    else
        install(newname, newline, newtype);
    ...
}

lookup(s)
char *s; {
    int i;
    extern struct {
        char    id[10];
        int     line;
        char    type;
        int     usage;
    } sym[ ];

    for( i=0; i<nsym; i++ )
        if( compar(s, sym[i].id) > 0 )
            return(i);

    return(-1);
}

compar(s1,s2)          /* return 1 if s1==s2, 0 otherwise */
char *s1, *s2; {
    while( *s1++ == *s2 )
        if( *s2++ == '\0' )
            return(1);

    return(0);
}

```

The declaration of the structure in `lookup` isn't needed if the external definition precedes its use in the same source file, as we shall see in a moment.

Now what if we want to use pointers?

```

struct symtag {
    char    id[10];
    int     line;
    char    type;
    int     usage;
} sym[100], *psym;

psym = &sym[0]; /* or p = sym; */

```

This makes `psym` a pointer to our kind of structure (the symbol table), then initializes it to point to the first element of `sym`.

Notice that we added something after the word `struct`: a ```tag"` called `symtag`. This puts a name on our structure definition so we can refer to it later without repeating the definition. It's not necessary but useful. In fact we could have said

```
struct  symtag {
    ... structure definition
};
```

which wouldn't have assigned any storage at all, and then said

```
struct  symtag  sym[100];
struct  symtag  *psym;
```

which would define the array and the pointer. This could be condensed further, to

```
struct  symtag  sym[100], *psym;
```

The way we actually refer to an member of a structure by a pointer is like this:

```
ptr -> structure-member
```

The symbol ``->'` means we're pointing at a member of a structure; ``->'` is only used in that context. `ptr` is a pointer to the (base of) a structure that contains the structure member. The expression `ptr->structure-member` refers to the indicated member of the pointed-to structure. Thus we have constructions like:

```
psym->type = 1;
psym->id[0] = 'a';
```

For more complicated pointer expressions, it's wise to use parentheses to make it clear who goes with what. For example,

```
struct { int x, *y; } *p;
p->x++ increments x
++p->x so does this!
(++p)->x increments p before getting x
*p->y++ uses y as a pointer, then increments it
*(p->y)++ so does this
*(p++)->y uses y as a pointer, then increments p
```

The way to remember these is that `->`, `.` (dot), `( )` and `[ ]` bind very tightly. An expression involving one of these is treated as a unit. `p->x`, `a[i]`, `y.x` and `f(b)` are names exactly as `abc` is.

If `p` is a pointer to a structure, any arithmetic on `p` takes into account the actual size of the structure. For instance, `p++` increments `p` by the correct amount to get the next element of the array of structures. But don't assume that the size of a structure is the sum of the sizes

of its members -- because of alignments of different sized objects, there may be "holes" in a structure.

Enough theory. Here is the lookup example, this time with pointers.

```
struct symtag {
    char    id[10];
    int     line;
    char    type;
    int     usage;
} sym[100];

main( ) {
    struct symtag *lookup( );
    struct symtag *psym;
    ...
    if( (psym = lookup(newname)) ) /* non-zero pointer */
        psym -> usage++;           /* means already there */
    else
        install(newname, newline, newtype);
    ...
}

struct symtag *lookup(s)
    char *s; {
    struct symtag *p;
    for( p=sym; p < &sym[nsym]; p++ )
        if( compar(s, p->id) > 0)
            return(p);
    return(0);
}
```

The function `compar` doesn't change: `'p->id'` refers to a string.

In `main` we test the pointer returned by `lookup` against zero, relying on the fact that a pointer is by definition never zero when it really points at something. The other pointer manipulations are trivial.

The only complexity is the set of lines like

```
struct symtag *lookup( );
```

This brings us to an area that we will treat only hurriedly; the question of function types. So far, all of our functions have returned integers (or characters, which are much the same). What do we do when the function returns something else, like a pointer to a structure? The rule is that any function that doesn't return an `int` has to say explicitly what it does return. The type information goes before the function name (which can make the name hard to see).

Examples:

```
char f(a)
    int a; {
        ...
    }
int *g( ) { ... }

struct symtag *lookup(s) char *s; { ... }
```

The function `f` returns a character, `g` returns a pointer to an integer, and `lookup` returns a pointer to a structure that looks like `symtag`. And if we're going to use one of these functions, we have to make a declaration where we use it, as we did in `main` above.

Notice the parallelism between the declarations

```
struct symtag *lookup( );
struct symtag *psym;
```

In effect, this says that `lookup( )` and `psym` are both used the same way - as a pointer to a structure -- even though one is a variable and the other is a function.

Initialization of Variables

An external variable may be initialized at compile time by following its name with an initializing value when it is defined. The initializing value has to be something whose value is known at compile time, like a constant.

```
int    x =    0;      /* "0" could be any constant */
int    *p    &y[1];  /* p now points to y[1] */
```

An external array can be initialized by following its name with a list of initializations enclosed in braces:

```
int    x[4] =    {0,1,2,3}; /* makes x[i] = i */
int    y[ ] =    {0,1,2,3}; /* makes y big enough for 4 values */
char    *msg =    "syntax error\n"; /* braces unnecessary here */
char    *keyword[ ]={
        "if",
        "else",
        "for",
        "while",
        "break",
        "continue",
        0
    };
```

This last one is very useful -- it makes keyword an array of pointers to character strings, with a zero at the end so we can identify the last element easily. A simple lookup routine could scan this until it either finds a match or encounters a zero keyword pointer:

```
lookup(str)          /* search for str in keyword[ ] */
char *str; {
    int i,j,r;
    for( i=0; keyword[i] != 0; i++) {
        for( j=0; (r=keyword[i][j]) == str[j] && r != '\0'; j++)
    );
        if( r == str[j] )
            return(i);
    }
    return(-1);
}
```

Scope Rules

A complete C program need not be compiled all at once; the source text of the program may be kept in several files, and previously compiled routines may be loaded from libraries. How do we arrange that data gets passed from one routine to another? We have already seen how to use function arguments and values, so let us talk about external data. Warning: the words declaration and definition are used precisely in this section; don't treat them as the same thing.

A major shortcut exists for making extern declarations. If the definition of a variable appears *before* its use in some function, no extern declaration is needed within the function. Thus, if a file contains

```
f1( ) { ... }
int foo;
f2( ) { ... foo = 1; ... }
f3( ) { ... if ( foo ) ... }
```

no declaration of `foo` is needed in either `f2` or `f3`, because the external definition of `foo` appears before them. But if `f1` wants to use `foo`, it has to contain the declaration

```
f1( ) {
    extern int foo;
    ...
}
```

This is true also of any function that exists on another file; if it wants `foo` it has to use an extern declaration for it. (If somewhere there is an extern declaration for something, there must also eventually be an external definition of it, or you'll get an "undefined symbol" message.)